

AIAC Assignment-10.2

Name:K.Chaitanya

Hallticket no:2303A51677

Batch no:B22

Task Description1--

ErrorDetectionandCorrection

Task:

Use AI to analyze a Python script and correct all syntax and logical errors.

Sample Input Code:

Def calculate(num):

Sum=0

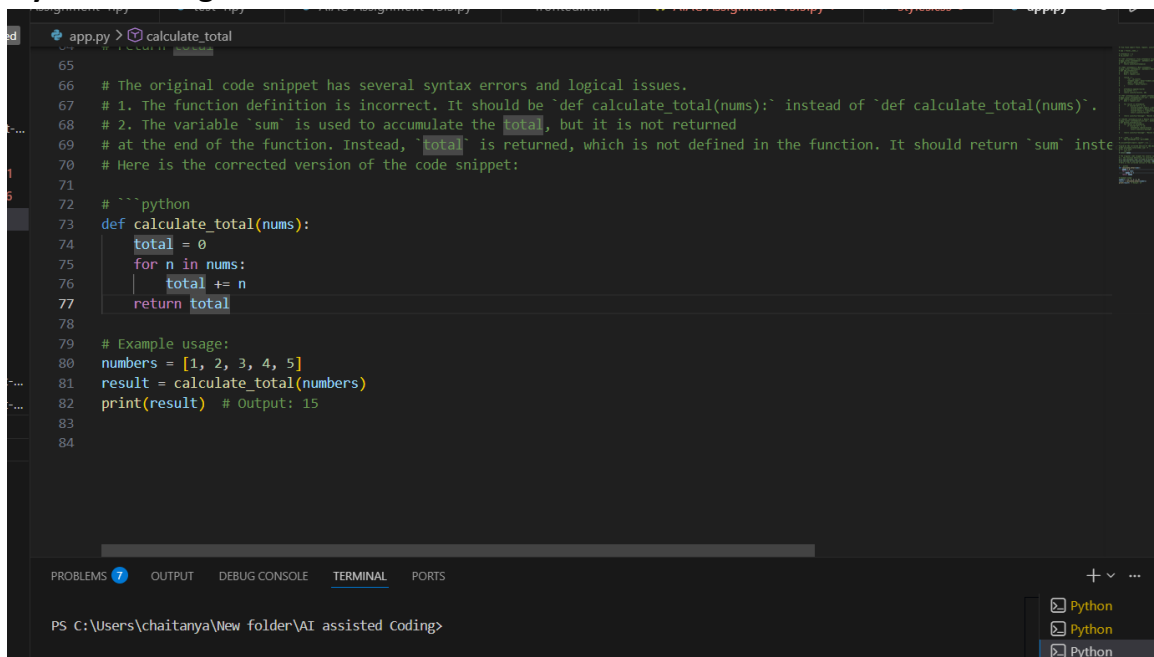
For n in nums:

Sum+=num

Return sum

Expected Output-1:

Corrected and executable Python code with brief explanations of the identified syntax and logic errors.



```
65
66 # The original code snippet has several syntax errors and logical issues.
67 # 1. The function definition is incorrect. It should be `def calculate_total(nums):` instead of `def calculate_total(nums)`.
68 # 2. The variable `sum` is used to accumulate the total, but it is not returned
69 # at the end of the function. Instead, `total` is returned, which is not defined in the function. It should return `sum` instead.
70 # Here is the corrected version of the code snippet:
71
72 # ```python
73 def calculate_total(nums):
74     total = 0
75     for n in nums:
76         total += n
77     return total
78
79 # Example usage:
80 numbers = [1, 2, 3, 4, 5]
81 result = calculate_total(numbers)
82 print(result) # Output: 15
83
84
```

Observation:

The original code snippet had a syntax error in the function definition and a logical error in the return statement. The corrected version defines the function properly and returns the correct variable that holds the total sum of the numbers.

Task Description -2

Use AI to refactor Python code to comply with standard coding style guidelines.

Sample Input Code:

```
def findSum(a,b):  
  
    return a+b  
  
print(findSum(5,10))
```

Expected Output-2:

Well-structured, consistently formatted Python code following standard style conventions.

```
def find_sum(a, b):  
    return a + b  
print(find_sum(5, 10))  
'''  
1. Function names should be in snake_case (e.g., `find_sum` instead of `findSum`).  
2. Use spaces around operators for better readability (e.g., `a + b` instead of `a+b`).
```

Observation:

Function names should be in snake_case

Like find_sum instead of findsum

Use spaces around operations for better readability

Like 'a + b' instead of 'a+b'

Task Description -3(Code Clarity Improvement)

Task:

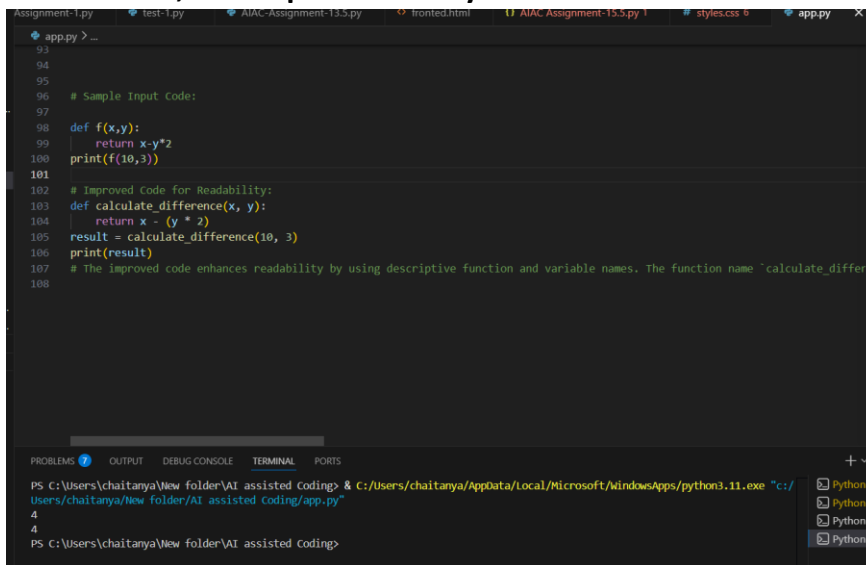
Use AI to improve code readability without changing its functionality.

Sample Input Code:

```
def f(x,y):  
  
    return x-y*2  
  
print(f(10,3))
```

Expected Output-

Python code rewritten with meaningful function and variable names, proper indentation, and improved clarity.



```
93  
94  
95  
96 # Sample Input Code:  
97  
98 def f(x,y):  
99     return x-y*2  
100 print(f(10,3))  
101  
102 # Improved Code for Readability:  
103 def calculate_difference(x, y):  
104     return x - (y * 2)  
105 result = calculate_difference(10, 3)  
106 print(result)  
107 # The improved code enhances readability by using descriptive function and variable names. The function name `calculate_differ  
108
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\chaitanya\New folder\AI assisted coding> & C:/Users/chaitanya/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/chaitanya/New folder/AI assisted coding/app.py"
4
4
PS C:\Users\chaitanya\New folder\AI assisted coding>

Obseravtion:

The improved code enhances readability by using descriptive function and variable names. The function name `calculate_difference` clearly indicates its purpose, and the variable `result` is more meaningful than just printing the function call directly. Additionally, the mathematical operation is made clearer by adding parentheses around `y * 2`, which emphasizes the order of operations.

Task Description -4(Structural

Refactoring) Task:

Use AI to refactor repetitive code into reusable

functions. Sample Input Code:

```
print("Hello
```

```
Ram")
```

```
print("Hello
```

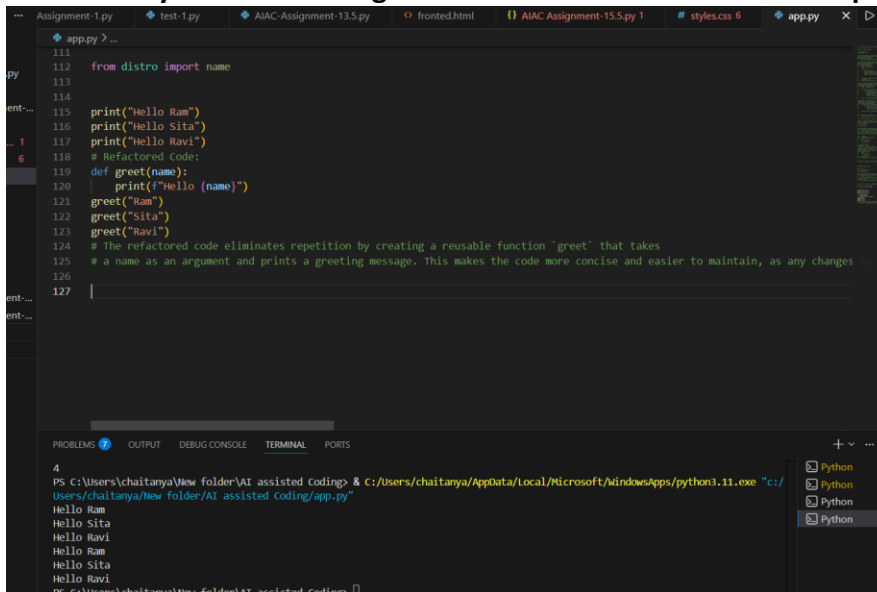
```
Sita")
```

```
print("Hello
```

```
Ravi")
```

Expected Output-4:

Modular Python code using reusable functions to eliminate repetition.



The screenshot shows a VS Code editor with a file named 'app.py'. The code in the file is as follows:

```
111
112 from distro import name
113
114
115 print("Hello Ram")
116 print("Hello Sita")
117 print("Hello Ravi")
118 # Refactored Code:
119 def greet(name):
120     print(f"Hello {name}")
121     greet("Ram")
122     greet("Sita")
123     greet("Ravi")
124 # The refactored code eliminates repetition by creating a reusable function 'greet' that takes
125 # a name as an argument and prints a greeting message. This makes the code more concise and easier to maintain, as any changes
126
127
```

The terminal at the bottom shows the output of the script:

```
4
PS C:\Users\chaitanya\New folder\AI assisted Coding> & C:/Users/chaitanya/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/chaitanya/New folder/AI assisted Coding/app.py"
Hello Ram
Hello Sita
Hello Ravi
Hello Ram
Hello Sita
Hello Ravi
PS C:\Users\chaitanya\New folder\AI assisted Coding>
```

Observation:

The refactored code eliminates repetition by creating a reusable function `greet` that takes a name as an argument and prints a greeting message. This makes the code more concise and easier to maintain, as any changes to the greeting format only need to be made in one place (the `greet` function) rather than in multiple print statements.

Task Description -5(Efficiency

Enhancement) Task:

Use AI to optimize Python code for better performance.

Sample Input Code:

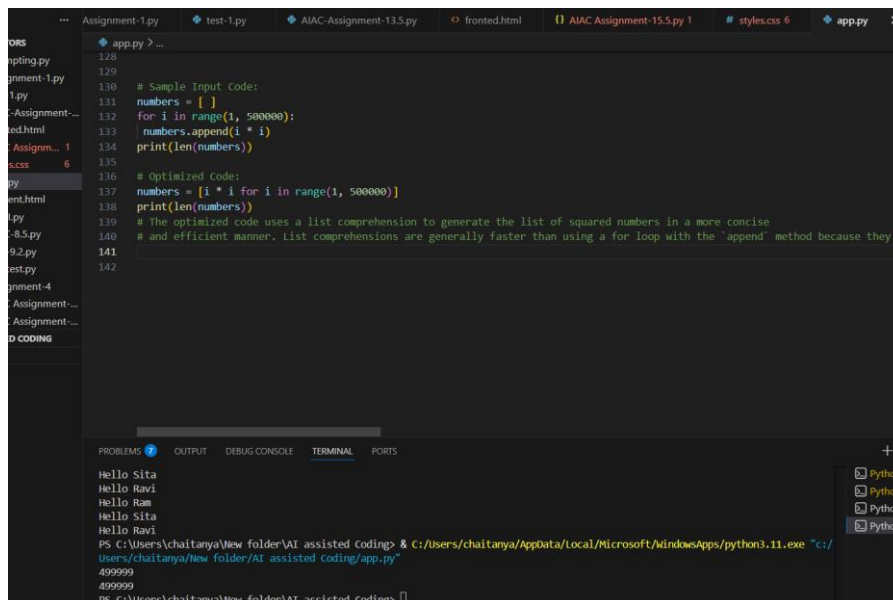
```
numbers = [ ]
```

```
for i in range(1, 500000):
```

```
    numbers.append(i * i)
```

```
print(len(numbers
```

```
)) Expected
```



```
128
129
130 # Sample Input Code:
131 numbers = [ ]
132 for i in range(1, 500000):
133     numbers.append(i * i)
134 print(len(numbers))
135
136 # Optimized Code:
137 numbers = [i * i for i in range(1, 500000)]
138 print(len(numbers))
139 # The optimized code uses a list comprehension to generate the list of squared numbers in a more concise
140 # and efficient manner. List comprehensions are generally faster than using a for loop with the 'append' method because they are
141
142
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Hello Sita
Hello Ravi
Hello Ram
Hello Sita
Hello Ravi

PS C:\Users\chaitanya\New folder\AI assisted coding> & C:\Users\chaitanya\AppData\Local\Microsoft\WindowsApps\python3.11.exe "C:/Users/chaitanya/New folder/AI assisted coding/app.py"
499999
PS C:\Users\chaitanya\New folder\AI assisted coding> |

Output-5:

Optimized Python code that achieves the same result with improved performance

The optimized code uses a list comprehension to generate the list of squared numbers in a more concise

and efficient manner. List comprehensions are generally faster than using a for loop with the `append` method because they are optimized in Python's implementation. This results in improved performance while achieving the same functionality.

